

PROJECT PLAN & ARCHITECTURE

Customer Support Ticket Classification

Automated Triage via Narrow AI

1. Problem Definition & Success Criteria

Customer support teams currently face significant bottlenecks when manually triaging incoming requests. The core objective of this initiative is to engineer a narrow AI classification model capable of automatically categorizing incoming customer support tickets into three distinct priority levels: **High** , **Medium** , and **Low** .

Target Accuracy

$\geq 85\%$

Overall Categorization Accuracy

High-Priority F1-Score

$\geq 80\%$

Ensuring urgent issues are captured

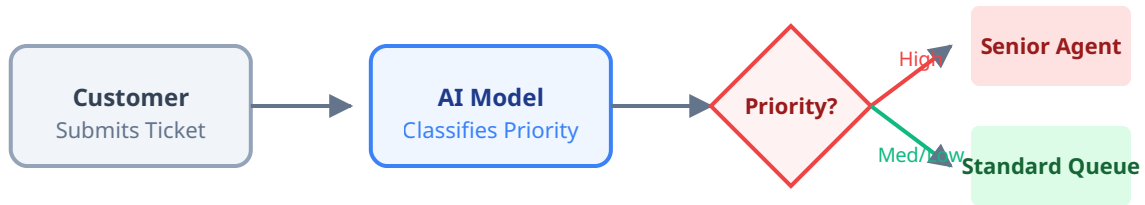
Success Criteria: The model must reliably achieve these metrics on holdout validation data. The overarching priority is preventing false negatives for critical issues (i.e., failing to flag a High-Priority ticket).

2. User Persona & Workflow

The Customer Support Triage Team

The primary users of this system are the Tier 1 Triage Agents and Team Leads. Currently, agents must read every incoming ticket to assess urgency, leading to delays for critical accounts. By integrating this model, the ticketing system will automatically route urgent items to senior agents within seconds of receipt.

Proposed Automated Workflow



3. Data Source Strategy

The model will be trained on a specialized, synthesized, and strictly anonymized dataset to circumvent PII compliance issues during the initial R&D phase.

- **Volume:** 1,000 historical customer support emails.
- **Schema:**
 - Ticket_Text: The natural language body of the email.
 - Priority_Label: Target classification (High, Medium, Low).

Implementation Idea: Given the small dataset size (1,000 records), advanced augmentation techniques (like synonym replacement or back-translation) should be explored to artificially expand the training set and prevent the model from overfitting to specific vocabulary.

4. System Constraints & Architecture Ideas

Deployment viability is heavily dependent on specific infrastructure and compliance boundaries. The architecture must conform to three strict constraints:

1. Compute Limitations (CPU Only)

The model must be lightweight enough to be fine-tuned and deployed on standard CPU hardware without requiring expensive GPU instances. **Recommendation:** Utilize fast traditional NLP models (like Support Vector Machines with TF-IDF) or highly quantized/distilled small language models (e.g., DistilBERT or fastText).

2. Latency Threshold

Classification inference must take less than **2 seconds per ticket**. This ensures smooth integration via webhooks into the live-ticketing platform without causing frontend timeout errors for agents.

3. Data Privacy & Compliance

No personally identifiable information (PII) can be transmitted to external APIs (e.g., OpenAI, Anthropic). The solution mandates a locally hosted, in-VPC model deployment.

5. Evaluation & Metrics Approach

The dataset will be partitioned into an 80% training set and a 20% validation holdout set to accurately simulate real-world performance. The model will be benchmarked against the following metrics:

Metric	Focus Area	Strategic Purpose
Accuracy	Overall performance	Measures the holistic correctness of categorization across all three classes. Target is $\geq 85\%$.
Recall (High)	Minimizing False Negatives	Critical to ensure urgent tickets are correctly identified. Missing a high priority ticket is penalized heavily.
Precision (High)	Minimizing False Positives	Ensures we don't flag non-urgent tickets as high priority, which would dilute the urgency and overwhelm senior agents.

Conceptual Confusion Matrix

Visualizing performance and category bleeding

		High	Medium	Low
Actual Class	High	88%	10%	2%
	Medium	12%	75%	13%
	Low	3%	15%	82%
		Predicted Class		

Note on Confusion Matrix: This visualization will be strictly monitored during evaluation to identify if the model struggles to differentiate between adjacent categories (e.g., confusing Medium with Low), allowing for targeted prompt or feature engineering.